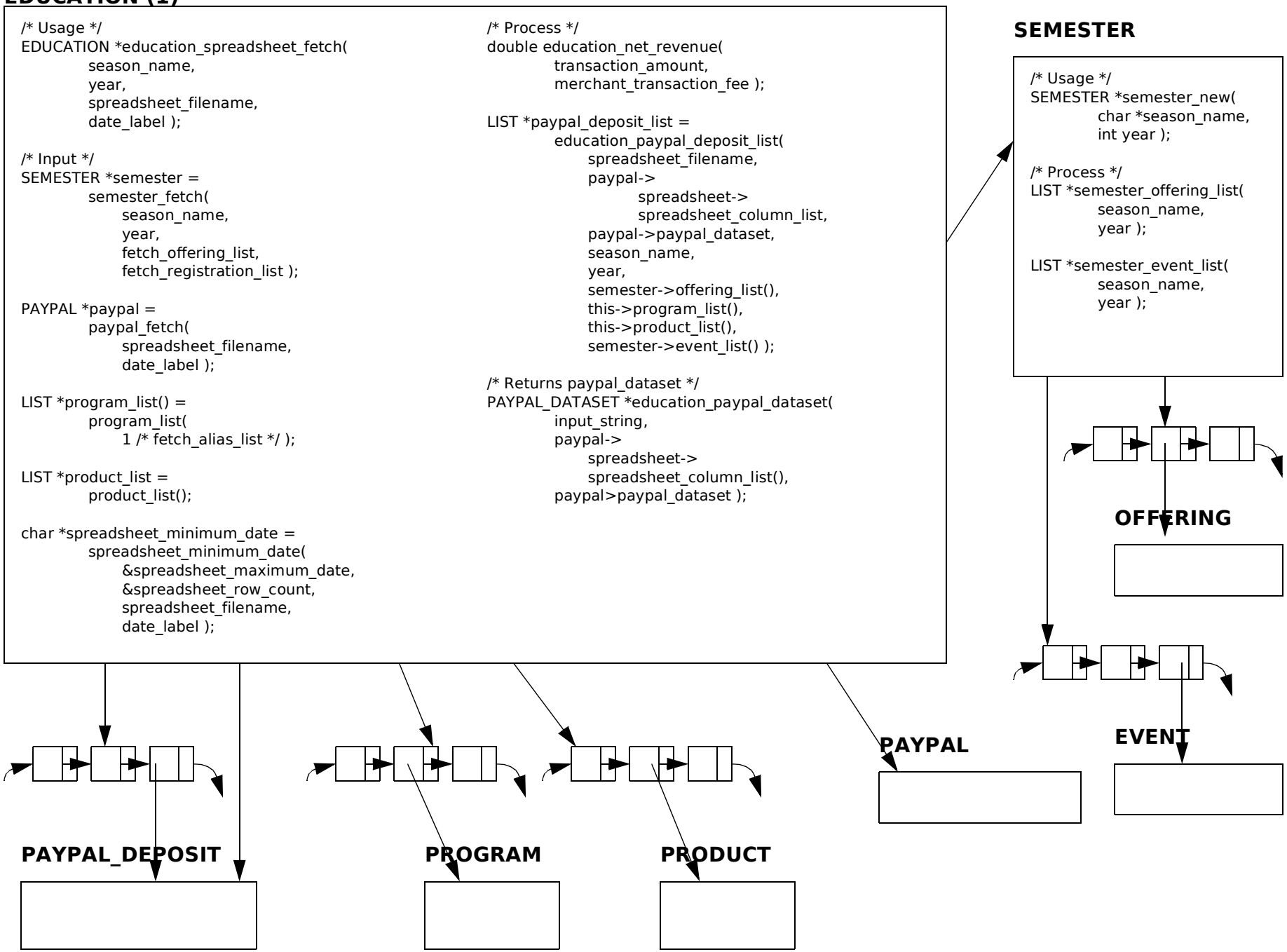
# Education Data Structure



## EDUCATION (1)

```
/* Usage */
EDUCATION *education_spreadsheet_fetch(
        season_name,
        year,
        spreadsheet_filename,
        date_label );

/* Input */
SEMESTER *semester =
        semester_fetch(
            season_name,
            year,
            fetch_offering_list,
            fetch_registration_list );

PAYPAL *paypal =
        paypal_fetch(
            spreadsheet_filename,
            date_label );

LIST *program_list() =
        program_list(
            1 /* fetch_alias_list */ );

LIST *product_list =
        product_list();

char *spreadsheet_minimum_date =
        spreadsheet_minimum_date(
            &spreadsheet_maximum_date,
            &spreadsheet_row_count,
            spreadsheet_filename,
            date_label );
```

```
/* Process */
double education_net_revenue(
        transaction_amount,
        merchant_transaction_fee );

LIST *paypal_deposit_list =
        education_paypal_deposit_list(
            spreadsheet_filename,
            paypal->
                spreadsheet->
                spreadsheet_column_list,
            paypal->paypal_dataset,
            season_name,
            year,
            semester->offering_list(),
            this->program_list(),
            this->product_list(),
            semester->event_list() );

/* Returns paypal_dataset */
PAYPAL_DATASET *education_paypal_dataset(
        input_string,
        paypal->
            spreadsheet->
            spreadsheet_column_list(),
        paypal>paypal_dataset );
```

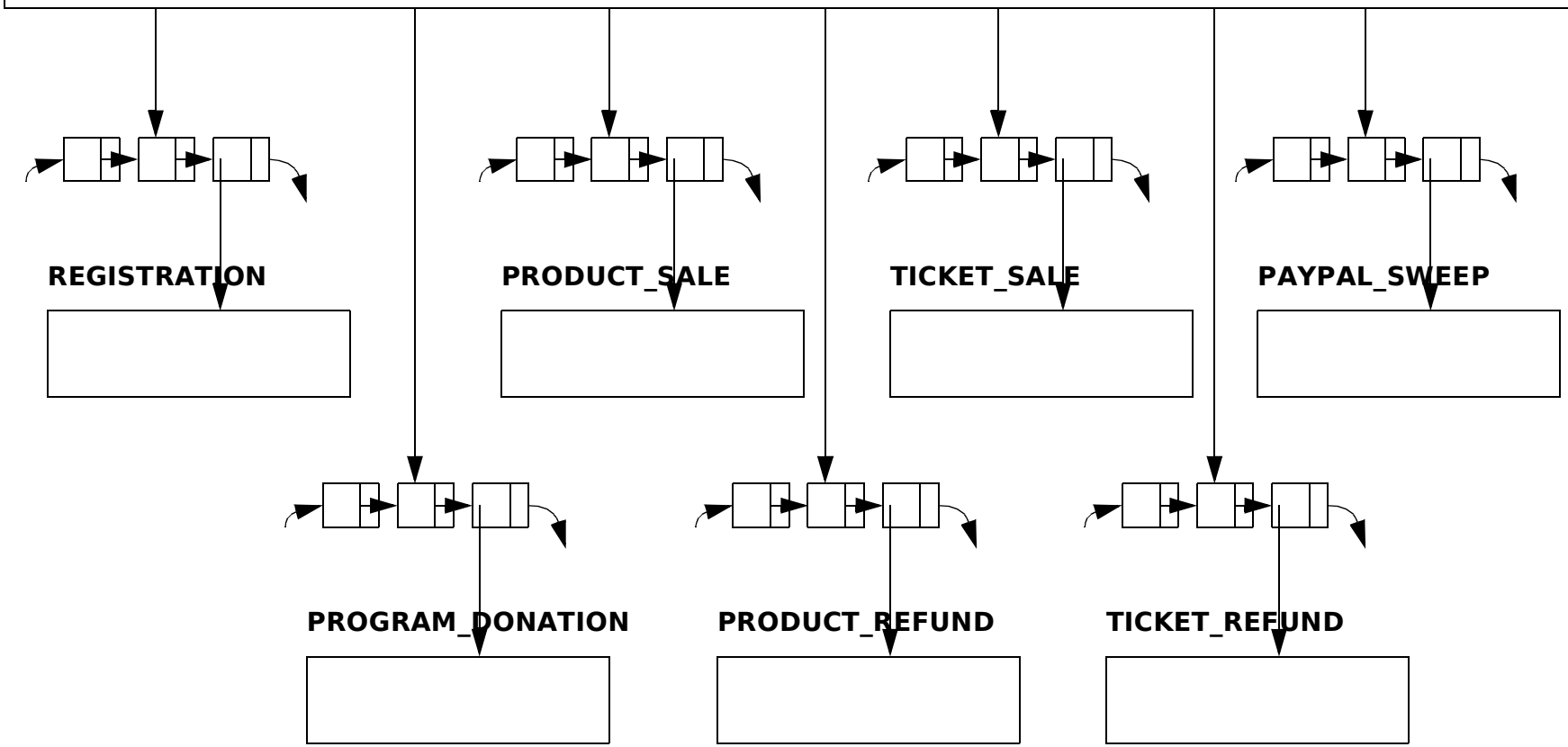


# Education Data Structure



## EDUCATION (2)

```
LIST *education_existing_program_donation_list(
        spreadsheet_minimum_date() );

LIST *education_existing_tuition_payment_list(
        spreadsheet_minimum_date() );

LIST *education_existing_tuition_refund_list(
        spreadsheet_minimum_date() );

LIST *education_existing_product_sale_list(
        spreadsheet_minimum_date() );

LIST *education_existing_product_refund_list(
        spreadsheet_minimum_date() );

LIST *education_existing_ticket_sale_list(
        spreadsheet_minimum_date() );

LIST *education_existing_ticket_refund_list(
        spreadsheet_minimum_date() );

LIST *education_existing_paypal_sweep_list(
        spreadsheet_minimum_date() );

/* Returns paypal_deposit_list */
LIST *education_paypal_existing_transaction_set(
        paypal_deposit_list,
        education_existing_program_donation_list(),
        education_existing_tuition_payment_list(),
        education_existing_tuiton_refund_list(),
        education_existing_product_sale_list(),
        education_existing_product_refund_list(),
        education_existing_ticket_sale_list(),
        education_existing_ticket_refund_list(),
        education_existing_paypal_sweep_list() );
```



## PAYPAL_DEPOSIT (1)

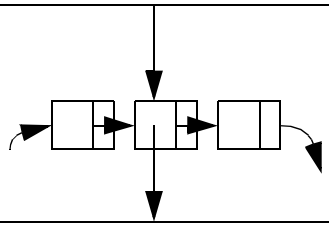
```
/* Usage */
PAYPAL_DEPOSIT *paypal_deposit_education(
        season_name,
        year,
        semester->offering_list(),
        program_list(),
        product_list(),
        semester->event_list(),
        paypal_dataset,
        row_number,
        /* Seek out existing registrations in this file */
        paypal_deposit_list() );

/* Input */
SEMESTER *semester =
        semester_new(
            season_name,
            year );
```

```
char *paypal_date_time;
double paypal_amount;
double transaction_fee;
int quantity;
char *from_email_address;
double account_balance;
char *transaction_ID;
char *invoice_number;
```

# Education Data Structure

## PAYPAL_DEPOSIT (2)

```
/* Process */
int row_number;

ENTITY *payor_entity =
        entity_full_name_entity(
            paypal_dataset->full_name_D );

LIST *paypal_item_list = paypal_item_list(
        paypal_item_entity_delimit(
            education_paypal_dataset()->
                item_title_P ),
        education_paypal_dataset()->
            transaction_type_E,
        education_paypal_allowed_list(
            offering_name_list()
            program_name_list(),
            program_alias_name_list(),
            product_name_list(),
            event_name_list() ) );

void paypal_deposit_set_paypal_item_expected_revenue(
        paypal_item_list(),
        semester_offering_list(),
        product_list(),
        semester_event_list() );

LIST *program_donation_list =
        program_donation_list_paypal();

LIST *tuition_payment_list =
        tuition_payment_list_paypal();

LIST *tuition_refund_list =
        tuition_refund_list_paypal();

LIST *product_sale_list =
        product_sale_list_paypal();

LIST *product_refund_list =
        product_refund_list_paypal();

LIST *ticket_sale_list =
        ticket_sale_list_paypal();

LIST *ticket_refund_list =
        ticket_refund_list_paypal();
```



## PAYPAL_ITEM

```
/* Usage */
LIST *paypal_item_list(
        entity_delimited_item_title_P,
        transaction_type_E,
        allowed_item_list,
        event_label_list );

/* Input */
double expected_revenue;

/* Process */
char *item_data;
boolean taken;

ENTITY *benefit_entity =
        entity_full_name_entity(
            item_title_P_full_name );

double paypal_item_value(
        paypal_amount,
        this->expected_revenue,
        paypal_item_expected_revenue_total(),
        paypal_item_nonexpected_revenue_length() );

double paypal_item_fee(
        paypal_amount,
        transaction_fee,
        paypal_item_value() );

double paypal_item_gain(
        paypal_amount,
        paypal_item_expected_revenue_total(),
        paypal_item_nonexpected_revenue_length(),
        paypal_item_expected_revenue_length() );

LIST *paypal_item_steady_state_list(
        paypal_item_list,
        paypal_amount,
        transaction_fee,
        paypal_item_expected_revenue_total(),
        paypal_item_nonexpected_revenue_length(),
        paypal_item_expected_revenue_length() );
```

# Education Data Structure

## PAYPAL_DEPOSIT (3)

```
double tuition_payment_total(
        tuition_payment_list );

double program_donation_total(
        program_donation_list );

double product_sale_total(
        product_sale_list );

double tuition_refund_total(
        tuition_refund_list );

double product_refund_total(
        product_refund_list );

double ticket_sale_total(
        ticket_sale_list );

double ticket_refund_total(
        ticket_refund_list );

PAYPAL_DEPOSIT *paypal_deposit_steady_state(
        this,
        semester_offering_list() );

void paypal_deposit_insert(
        payor_entity->full_name,
        payor_entity->street_address,
        paypal_date_time,
        row_number,
        season_name,
        year,
        paypal_amount,
        transaction_fee,
        net_revenue,
        tuition_payment_total(),
        program_donation_total(),
        ticket_sale_total(),
        product_sale_total(),
        sweep_amount(),
        tuition_refund_total(),
        ticket_refund_total(),
        product_refund_total(),
        quantity,
        from_email_address,
        account_balance,
        transaction_ID,
        invoice_number );

PAYPAL_SWEEP *paypal_sweep(
        paypal_item_list(),
        paypal_deposit->paypal_date_time );

boolean existing_transaction =
        paypal_deposit_existing_transaction(
            paypal_deposit,
            education_existing_program_donation_list(),
            education_existing_tuition_payment_list(),
            education_existing_tuition_refund_list(),
            education_existing_product_sale_list(),
            education_existing_product_refund_list(),
            education_existing_ticket_sale_list(),
            education_existing_ticket_refund_list(),
            education_existing_paypal_sweep_list() );
```

## PAYPAL_SWEEP

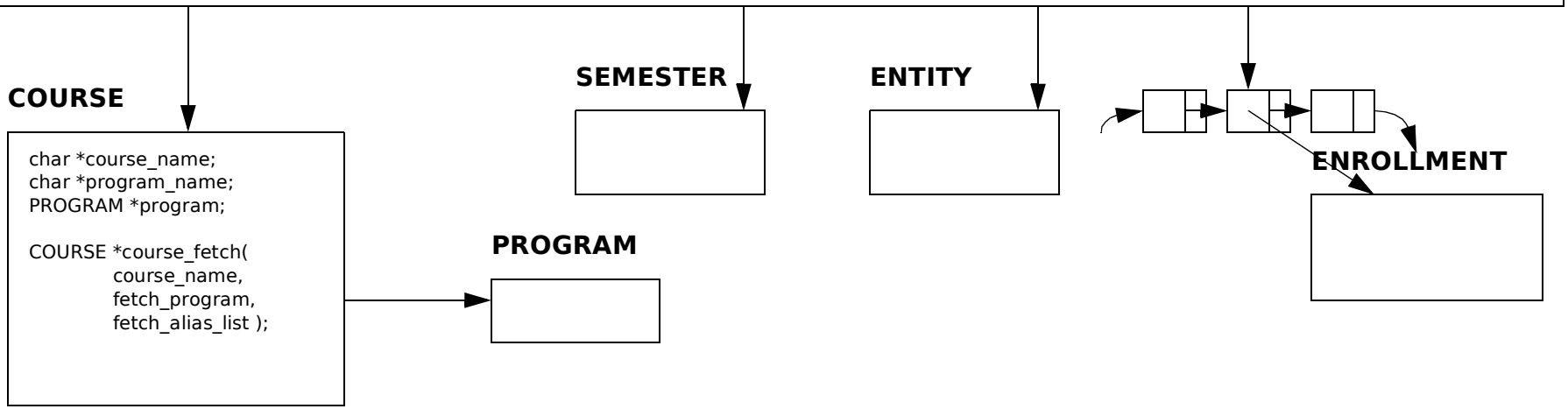
```
/* Usage */
PAYPAL_SWEEP *paypal_sweep_fetch(
        payor_full_name,
        payor_street_address,
        paypal_date_time,
        fetch_transaction );

char *payor_full_name;
char *payor_street_address;
char *paypal_date_time;
double sweep_amount;

/* Process */
TRANSACTION *paypal_sweep_transaction(
        payor_full_name,
        payor_street_address,
        paypal_date_time,
        sweep_amount,
        paypal_sweep_cash_account(),
        self_cash_account() );
```

## TRANSACTION

# Education Data Structure

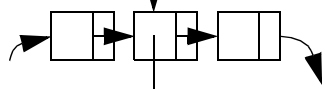
## OFFERING

```
/* Usage */
OFFERING *offering_fetch(
        course_name,
        season_name,
        year,
        fetch_course,
        fetch_program,
        fetch_enrollment_list );

/* Input */
COURSE *course;
SEMESTER *semester;
LIST *offering_enrollment_list;
double course_price;
int class_capacity;
char *revenue_account;

ENTITY *instructor_entity =
        entity_new(
            instructor_full_name;
            instructor_street_address );

LIST *offering_enrollment_list(
        course_name,
        season_name,
        year );

/* Process */
OFFERING *offering_parse(
        input,
        fetch_course,
        fetch_enrollment_list )

int offering_enrollment_count(
        offering_enrollment_list() )
{
        return length( A );
}

int offering_capacity_available(
        class_capacity,
        offering_enrollment_count() )
{
        return A - B;
}

OFFERING *offering_steady_state(
        this,
        offering_enrollment_list() );

/* Output */
void offering_update(
        offering_enrollment_count(),
        offering_capacity_available(),
        course_name,
        season_name,
        year );
```



**PROGRAM**

```
/* Usage */
PROGRAM *program_fetch(
        program_name,
        fetch_alias_list );

/* Input */
char *program_name;
char *revenue_account;

/* Process */
LIST *program_list(
        fetch_alias_list );

LIST *program_alias_list(
        program_name );

char *program_seek_name(
        program_name,
        program_list() );

PROGRAM *program_list_seek(
        program_alias_name,
        program_list() );

void program_donation_total(
        program_name );
```

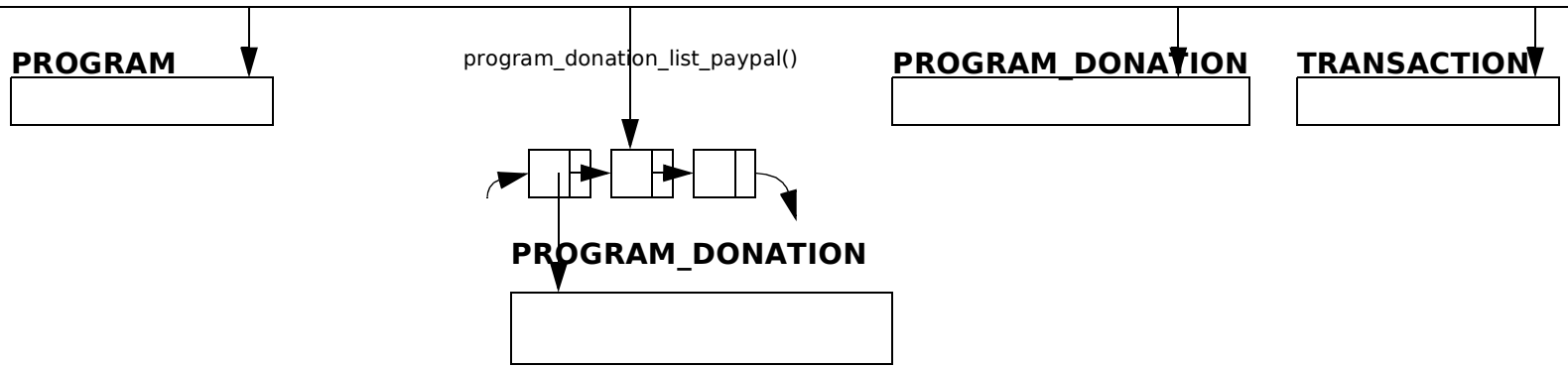


**PROGRAM_ALIAS**

```
char *program_name;
char *alias_name;
```

## PROGRAM_DONATION

```
/* Usage */
PROGRAM_DONATION *program_donation_fetch(
        program_name,
        payor_full_name,
        payor_street_address,
        season_name,
        year,
        payment_date_time,
        fetch_program,
        fetch_transaction );

/* Input */
char *program_name;
PROGRAM *program;
ENTITY *payor_entity;
char *payment_date_time;
double donation_amount;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double net_payment_amount =
        education_net_payment_amount(
            donation_amount,
            merchant_fees_expense );

PROGRAM_DONATION
*program_donation_steady_state(
        this,
        donation_amount,
        merchant_fees_expense );

PROGRAM_DONATION *program_donation_parse(
        input,
        fetch_program );

LIST *program_donation_list_paypal(
        paypal_deposit->payor_entity,
        paypal_deposit->paypal_date_time,
        paypal_item_list(),
        program_list() );

PROGRAM_DONATION *program_donation_paypal(
        payor_entity,
        paypal_date_time,
        paypal_item->item_value,
        paypal_item->item_fee,
        program );

TRANSACTION *program_donation_transaction(
        *seconds_to_add,
        payor_entity->full_name,
        payor_entity->street_address,
        payment_date_time,
        program_name,
        donation_amount,
        merchant_fees_expense,
        education_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        program->revenue_account );

void program_payment_update(
        education_net_payment_amount(),
        program_donation_transaction()->
            transaction_date_time,
        program_name,
        payor_entity->full_name,
        payor_entity->street_address,
        season_name,
        year,
        payment_date_time );
```



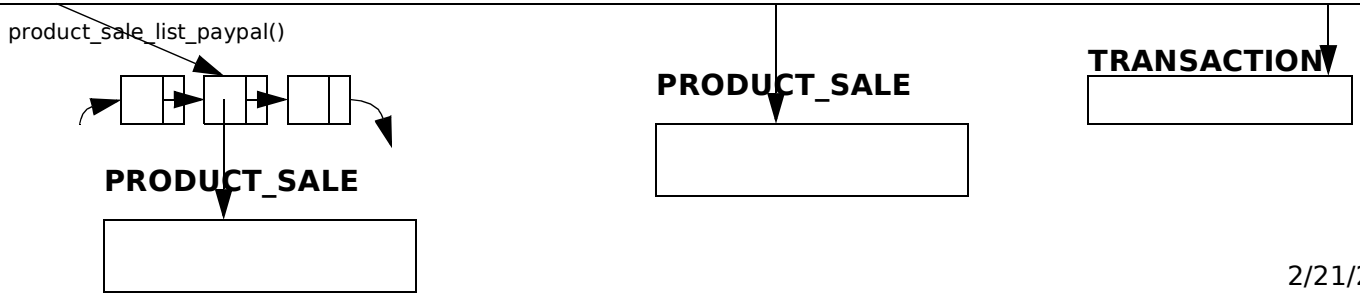
# Education Data Structure

## PRODUCT

```
/* Usage */
PRODUCT *product_fetch(
        char *product_name,
        boolean fetch_program,
        boolean fetch_sale_list,
        boolean fetch_refund_list );

/* Input */
PROGRAM *program;
double retail_price;
LIST *product_sale_list();
LIST *product_refund_list();

/* Process */
double product_sale_total(
        product_sale_list() );
double product_refund_total(
        product_refund_list() );

void product_update(
        product_sale_total(),
        product_refund_total(),
        product_name );
```

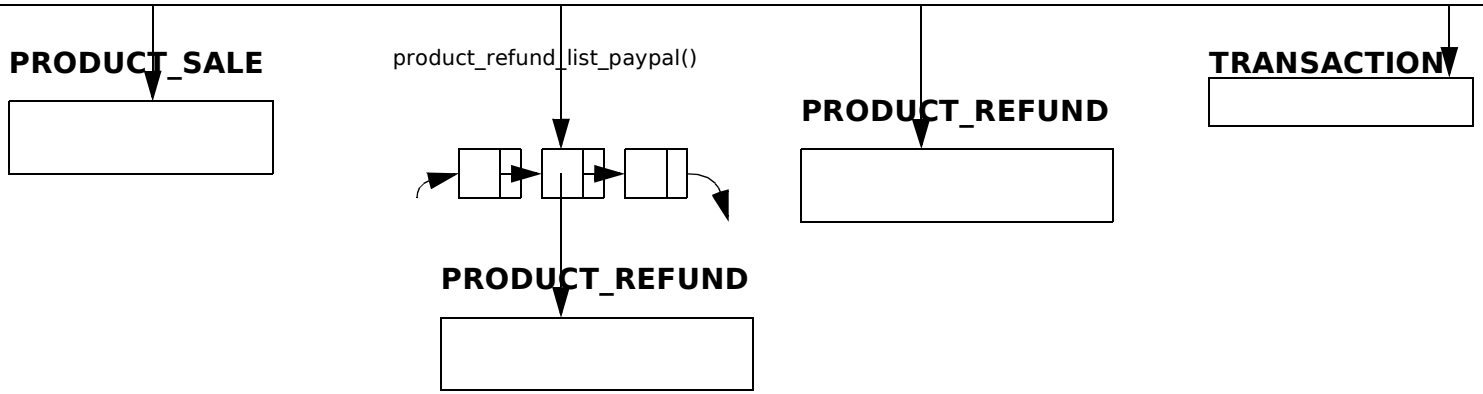
## PRODUCT_SALE

```
/* Usage */
PRODUCT_SALE *product_sale_fetch(
        product_name,
        payor_full_name,
        payor_street_address,
        sale_date_time,
        fetch_product,
        fetch_transaction );

/* Input */
char *product_name;
ENTITY *payor_entity;
char *sale_date_time;
PRODUCT *product;
int quantity;
double retail_price;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double product_sale_extended_price(
        quantity,
        retail_price );

double net_payment_amount =
        education_net_payment_amount(
            product_sale_extended_price(),
            merchant_fees_expense );

PRODUCT_SALE *product_sale_parse(
        input,
        fetch_product );

PRODUCT_SALE *product_sale_steady_state(
        this,
        quantity,
        retail_price,
        merchant_fees_expense );

LIST *product_sale_list_paypal(
        paypal_deposit->payor_entity,
        paypal_deposit->paypal_date_time,
        paypal_item_list(),
        product_list() );

PRODUCT_SALE *product_sale_paypal(
        payor_entity,
        paypal_date_time,
        paypal_item->item_value,
        paypal_item->item_fee,
        product );

TRANSACTION *product_sale_transaction(
        *seconds_to_add,
        payor_entity->full_name,
        payor_entity->street_address,
        sale_date_time,
        product_name,
        program_name,
        product_sale_extended_price(),
        merchant_fees_expense,
        education_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        product->revenue_account );

void product_sale_update(
        product_sale_extended_price(),
        education_net_payment_amount(),
        product_sale_transaction()->
            transaction_date_time,
        product_name,
        payor_entity->full_name,
        payor_entity->street_address,
        sale_date_time );
```



## PRODUCT_REFUND

```
/* Usage */
PRODUCT_REFUND *product_refund_fetch(
        product_name,
        payor_full_name,
        payor_street_address,
        sale_date_time,
        refund_date_time,
        fetch_sale,
        fetch_product,
        fetch_transaction );

/* Input */
char *product_name;
ENTITY *payor_entity;
char *sale_date_time;
char *refund_date_time;
PRODUCT_SALE *product_sale;
double refund_amount;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double net_refund_amount =
        education_net_refund_amount(
            refund_amount,
            merchant_fees_expense );

PRODUCT_REFUND *product_refund_parse(
        input,
        fetch_sale,
        fetch_product );

PRODUCT_REFUND *product_refund_steady_state(
        this,
        refund_amount,
        merchant_fees_expense );

LIST *product_refund_list_paypal(
        paypal_deposit->payor_entity,
        paypal_deposit->paypal_date_time,
        paypal_item_list(),
        product_list() );

PRODUCT_REFUND *product_refund_paypal(
        payor_entity,
        paypal_date_time,
        paypal_item->item_value,
        paypal_item->item_fee,
        product );

TRANSACTION *product_refund_transaction(
        *seconds_to_add,
        payor_entity->full_name,
        payor_entity->street_address,
        refund_date_time,
        product_name,
        this->product_sale->program_name,
        refund_amount,
        merchant_fees_expense,
        education_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        product->revenue_account );

void product_refund_update(
        education_net_refund_amount(),
        product_refund_transaction()->
            transaction_date_time,
        product_name,
        payor_entity->full_name,
        payor_entity->street_address,
        sale_date_time,
        refund_date_time );
```

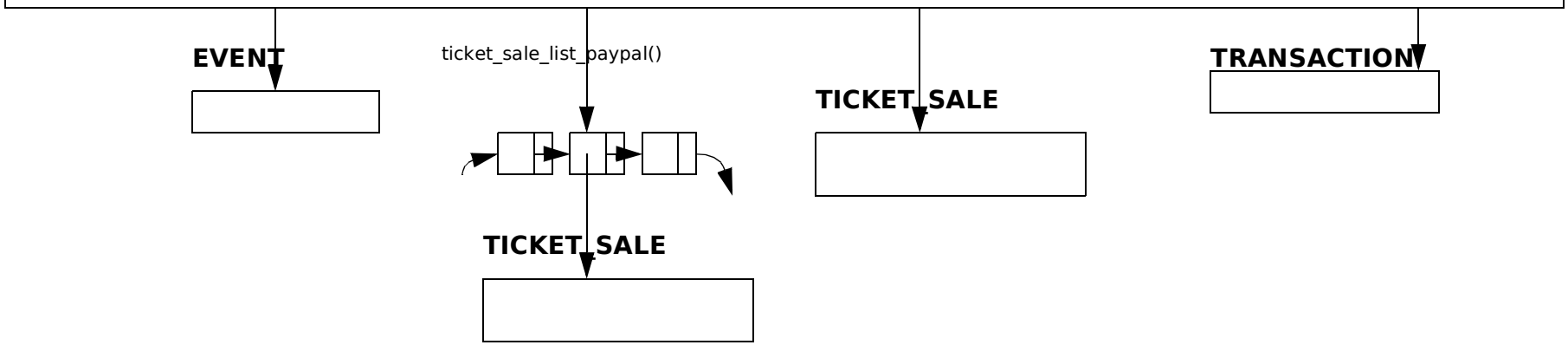


## EVENT

```
/* Usage */
EVENT *event_fetch(
        char *program_name,
        char *event_date,
        char *event_time,
        boolean fetch_program,
        boolean fetch_venue,
        boolean fetch_sale_list,
        boolean fetch_refund_list );

/* Input */
char *program_name;
char *event_date;
char *event_time;
char *venue_name;
char *venue_street_address;
PROGRAM *program;
VENUE *venue;
double ticket_price;
char *season_name;
int year;
LIST *ticket_sale_list();
LIST *ticket_refund_list();

/* Process */
char *event_paypal_display(
        program_name,
        event_date,
        event_time );

int ticket_sale_count =
        ticket_sale_count(
            ticket_sale_list );

double ticket_sale_total =
        ticket_sale_total(
            ticket_sale_list );
int ticket_refund_count =
        ticket_refund_count(
            ticket_sale_list );

double ticket_refund_total =
        ticket_refund_total(
            ticket_refund_list );

int event_capacity_available(
        this->venue->capacity,
        ticket_sale_count(),
        ticket_refund_count() );

void event_fetch_update(
        program_name,
        event_date,
        event_time );
```

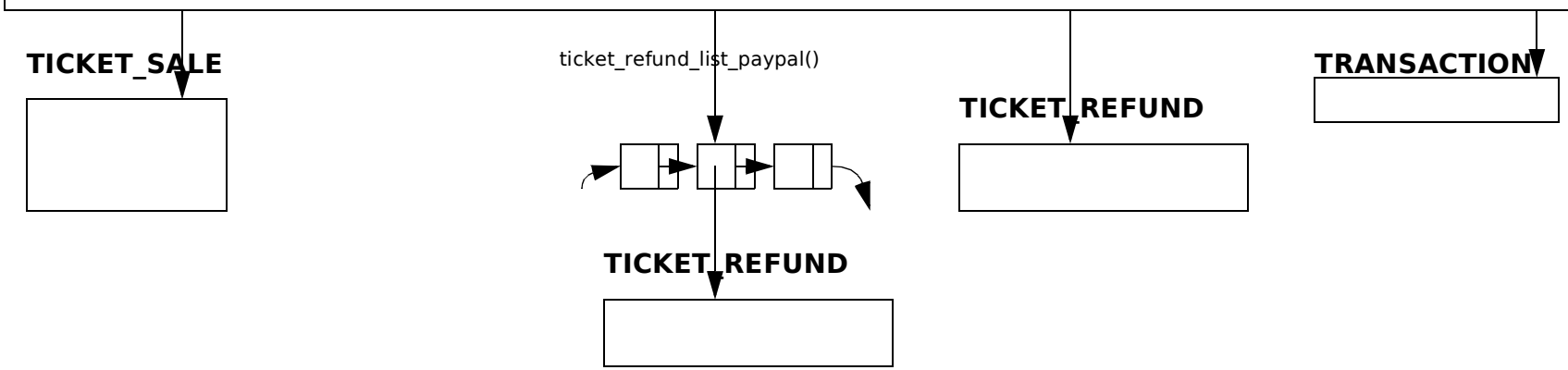
## TICKET_SALE

```
/* Usage */
TICKET_SALE *ticket_sale_fetch(
        program_name,
        event_date,
        event_time,
        payor_full_name,
        payor_street_address,
        sale_date_time,
        fetch_event,
        fetch_transaction );

/* Input */
char *program_name;
char *event_date;
char *event_time;
ENTITY *payor_entity;
char *sale_date_time;
EVENT *event;
int quantity;
double ticket_price;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double ticket_sale_extended_price(
        quantity,
        ticket_price );

double net_payment_amount =
        education_net_payment_amount(
            ticket_sale_extended_price(),
            merchant_fees_expense );

TICKET_SALE *ticket_sale_parse(
        input,
        fetch_event );

TICKET_SALE *ticket_sale_steady_state(
        this,
        quantity,
        ticket_price,
        merchant_fees_expense );

LIST *ticket_sale_list_paypal(
        paypal_deposit->payor_entity,
        paypal_deposit->paypal_date_time,
        paypal_item_list(),
        semester_event_list(),
        paypal_amount );

TICKET_SALE *ticket_sale_paypal(
        payor_entity,
        paypal_date_time,
        paypal_item->item_value,
        paypal_item->item_fee,
        event,
        paypal_amount,
        paypal_item_list_length );

TRANSACTION *ticket_sale_transaction(
        *seconds_to_add,
        payor_entity->full_name,
        payor_entity->street_address,
        sale_date_time,
        event->program->program_name,
        ticket_sale_extended_price(),
        merchant_fees_expense,
        education_net_payment_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        event->revenue_account );

void ticket_sale_update(
        ticket_sale_extended_price(),
        education_net_payment_amount(),
        ticket_sale_transaction()->
            transaction_date_time,
        program_name,
        event_date,
        event_time,
        payor_entity->full_name,
        payor_entity->street_address,
        sale_date_time );
```



## TICKET_REFUND

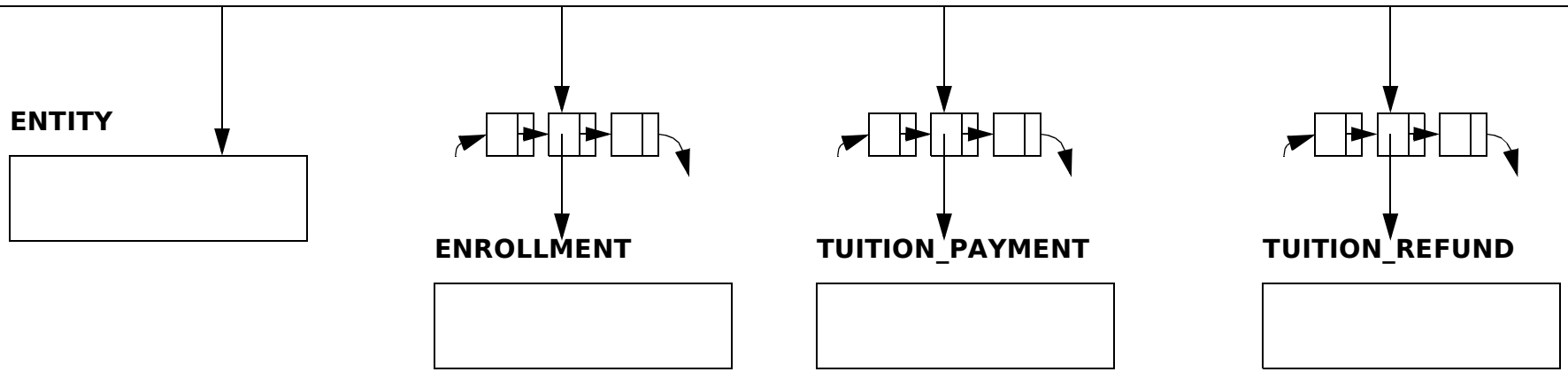
```
/* Usage */
TICKET_REFUND *ticket_refund_fetch(
        program_name,
        event_date,
        event_time,
        payor_full_name,
        payor_street_address,
        sale_date_time,
        refund_date_time,
        fetch_sale,
        fetch_transaction );

/* Input */
char *program_name;
char *event_date;
char *event_time;
ENTITY *payor_entity;
char *sale_date_time;
char *refund_date_time;
TICKET_SALE *ticket_sale;
double refund_amount;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double net_refund_amount =
        education_net_refund_amount(
            refund_amount,
            merchant_fees_expense );

TICKET_REFUND *ticket_refund_parse(
        input,
        fetch_sale );

TICKET_REFUND *ticket_refund_steady_state(
        this,
        refund_amount,
        merchant_fees_expense );

LIST *ticket_refund_list_paypal(
        paypal_deposit->payor_entity,
        paypal_deposit->paypal_date_time,
        paypal_item_list(),
        semester_event_list() );

TUITION_REFUND *ticket_refund_paypal(
        payor_entity,
        paypal_date_time,
        paypal_item->item_value,
        paypal_item->item_fee,
        event );

TRANSACTION *ticket_refund_transaction(
        *seconds_to_add,
        payor_entity->full_name,
        payor_entity->street_address,
        refund_date_time,
        this->
            ticket_sale->
            event->
            event_name,
        this->
            ticket_sale->
            event->
            program->
            program_name,
        refund_amount,
        merchant_fees_expense,
        education_net_refund_amount(),
        entity_self_paypal_cash_account_name(),
        account_fees_expense(),
        this->
            ticket_sale->
            event->
            revenue_account );

void ticket_refund_update(
        education_net_refund_amount(),
        ticket_refund_transaction()->
            transaction_date_time,
        program_name,
        event_date,
        event_time,
        payor_entity->full_name,
        payor_entity->street_address,
        sale_date_time,
        refund_date_time );
```



# Education Data Structure

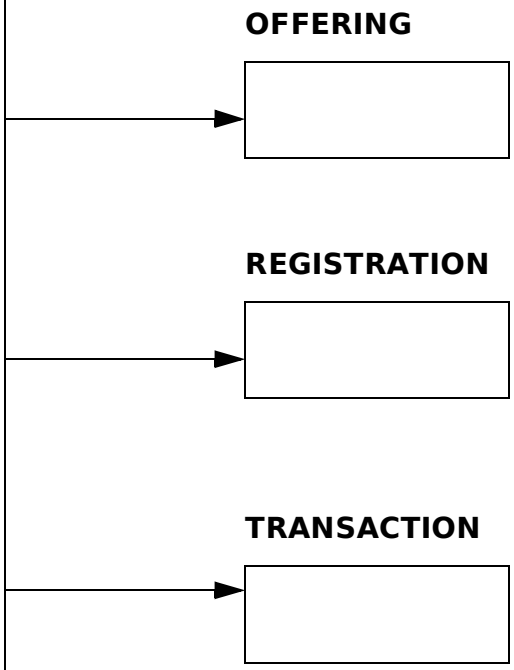
## REGISTRATION

```
/* Usage */
REGISTRATION *registration_fetch(
        student_full_name,
        student_street_address,
        season_name,
        year,
        fetch_enrollment_list,
        fetch_offering,
        fetch_course,
        fetch_program,
        fetch_tuition_payment_list,
        fetch_tuition_refund_list );

/* Input */
ENTITY *student_entity;
LIST *registration_enrollment_list;
char *season_name;
int year;

/* Process */
LIST *registration_tuition_payment_list(
        student_entity->full_name,
        student_entity->street_address,
        season_name,
        year );

LIST *registration_tuition_refund_list(
        student_entity->full_name,
        student_entity->street_address,
        season_name,
        year );

char *registration_offering_program_name(
        registration_enrollment_list );

double registration_tuition(
        registration_enrollment_list,
        semester_offering_list() );

double registration_tuition_payment_total(
        registration_tuition_payment_list )
{
        return sum(A->
                tuition_payment->
                payment_amount );
}

double registration_tuition_refund_total(
        tuition_refund_list );

double registration_invoice_amount_due(
        registration_tuition(),
        registration_tuition_payment_total(),
        registration_tuition_refund_total() );

void registration_update(
        registration_tuition(),
        registration_tuition_payment_total(),
        registration_tuition_refund_total(),
        registration_invoice_amount_due(),
        student_entity->full_name,
        student_entity->street_address,
        season_name,
        year );
```



## ENROLLMENT

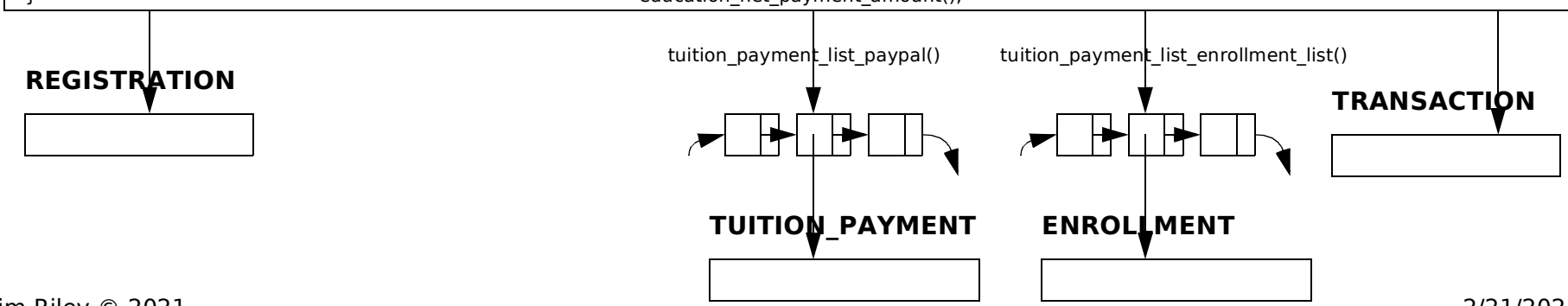
```
/* Usage */
ENROLLMENT *enrollment_fetch(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        fetch_offering,
        fetch_course,
        fetch_program,
        fetch_registration,
        fetch_transaction );

/* Input */
REGISTRATION *registration;
OFFERING *offering;

/* Process */
char *enrollment_primary_where(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year );

TRANSACTION *enrollment_transaction(
        *seconds_to_add,
        student_full_name,
        street_address,
        this->registration->registration_date_time,
        this->offering->course->program_name,
        this->offering->course->course_price,
        account_receivable(),
        offering_revenue_account() );

char *enrollment_list_first_program_name(
        enrollment_list );
```



# Education Data Structure

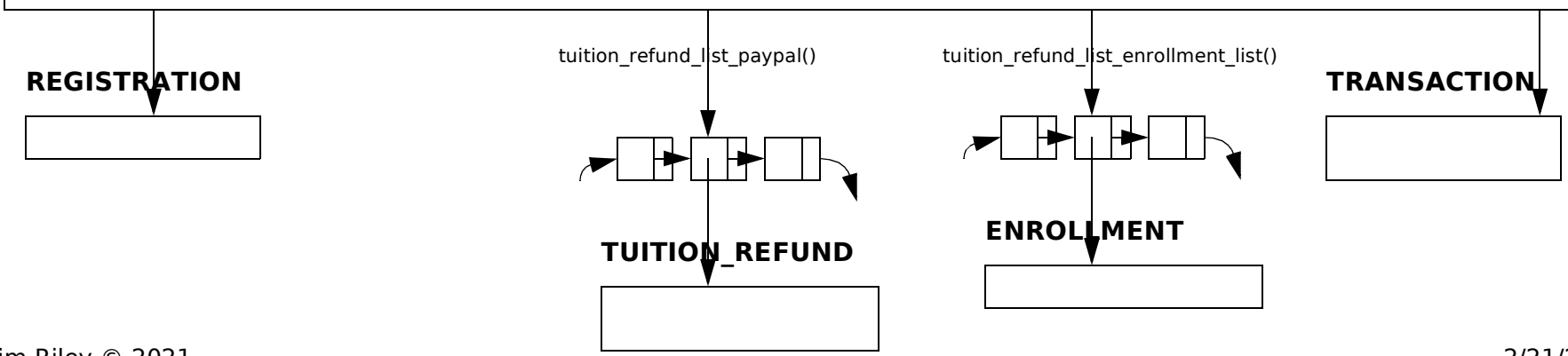
## TUITION_PAYMENT

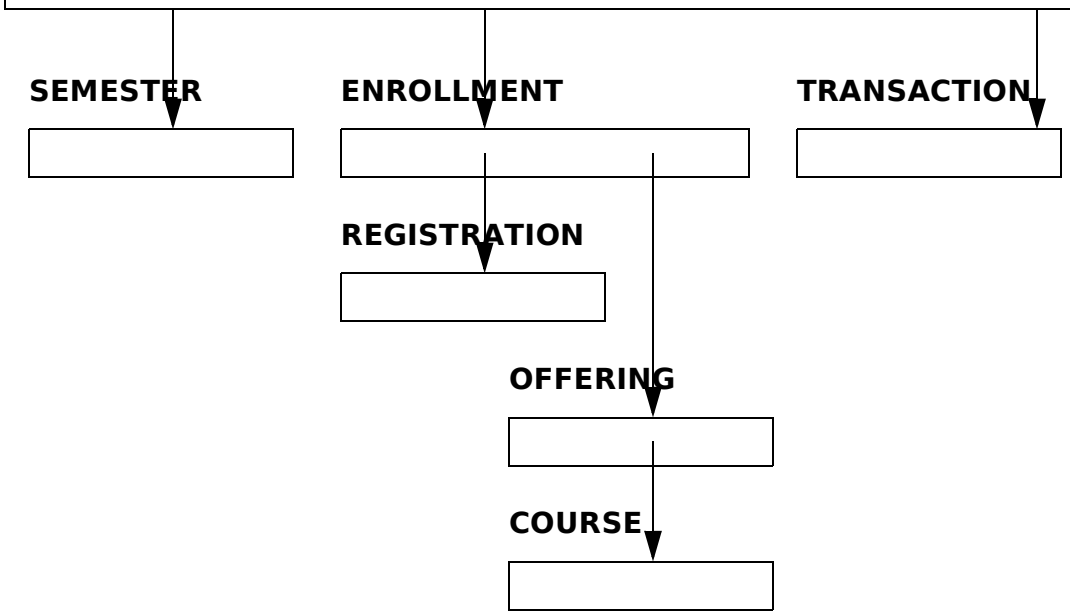
```
/* Usage */
TUITION_PAYMENT *tuition_payment_fetch(
        student_full_name,
        street_address,
        season_name,
        year,
        payor_full_name,
        payor_street_address,
        payment_date_time,
        boolean fetch_registration,
        boolean fetch_enrollment_list,
        boolean fetch_offering,
        boolean fetch_course,
        boolean fetch_program,
        boolean fetch_transaction );

/* Input */
REGISTRATION *registration;
ENTITY *payor_entity;
char *payment_date_time;
double payment_amount;
double merchant_fees_expense;
char *paypal_date_time;

/* Process */
double net_payment_amount =
        education_net_payment_amount(
            payment_amount,
            merchant_fees_expense );

double tuition_payment_total(
        tuition_payment_list )
{
        return sum(A->
                tuition_payment->
                payment_amount );
}

TUITION_PAYMENT *tuition_payment_steady_state(
        this,
        payment_amount,
        merchant_fees_expense,
        semester_offering_list() );

LIST *tuition_payment_list_paypal(
        season_name,
        year,
        paypal_deposit->payor_entity,
        paypal_deposit->paypal_date_time,
        paypal_item_list(),
        semester_offering_list() );

TUITION_PAYMENT *tuition_payment_paypal(
        season_name,
        year,
        paypal_item->benefit_entity
            /* student_entity */,
        payor_entity,
        paypal_date_time,
        paypal_item->item_value,
        paypal_item->item_fee,
        offering );

TRANSACTION *tuition_payment_transaction(
        *seconds_to_add,
        payor_entity->full_name,
        payor_entity->street_address,
        payment_date_time,
        offering->
            course->
            program_name,
        payment_amount,
        merchant_fees_expense,
        education_net_payment_amount(),
        account_cash(),
        account_receivable(),
        account_fees_expense() );

void tuition_payment_update(
        education_net_payment_amount(),
        tuition_payment_transaction()->
            transaction_date_time,
        student_entity->full_name,
        student_entity->street_address,
        season_name,
        year,
        payor_entity->full_name,
        payor_entity->street_address,
        payment_date_time );

LIST *tuition_payment_list_enrollment_list(
        tuition_payment_list );

LIST *tuition_payment_enrollment_list(
        tuition_payment );
```



## TUITION_REFUND

```
/* Usage */
TUITION_REFUND *tuition_refund_fetch(
        student_full_name,
        street_address,
        season_name,
        year,
        payor_full_name,
        payor_street_address,
        payment_date_time,
        refund_date_time,
        boolean fetch_registration,
        boolean fetch_enrollment_list,
        boolean fetch_offering,
        boolean fetch_course,
        boolean fetch_program );

/* Input */
REGISTRATION *registration;
ENTITY *payor_entity;
char *refund_date_time;
double refund_amount;
double merchange_fees_expense;
char *paypal_date_time;

/* Process */
double net_refund_amount =
        education_net_refund_amount(
            refund_amount,
            merchant_fees_expense );

double tuition_refund_total(
        tuition_refund_list )
{
        return sum(A->
                tuition_refund->
                refund_amount );
}

TUITION_REFUND *tuition_refund_steady_state(
        this,
        refund_amount,
        merchant_fees_expense );

LIST *tuition_refund_list_paypal(
        season_name,
        year,
        paypal_deposit->payor_entity,
        paypal_deposit->paypal_date_time,
        paypal_item_list(),
        semester_offering_list() );

TUITION_REFUND *tuition_refund_paypal(
        season_name,
        year,
        paypal_item->benefit_entity
            /* student_entity */,
        payor_entity,
        paypal_date_time,
        paypal_item->item_value,
        paypal_item->item_fee,
        offering );

TRANSACTION *tuition_refund_transaction(
        *seconds_to_add,
        payor_entity->full_name,
        payor_entity->street_address,
        refund_date_time,
        offering->course->program_name,
        refund_amount,
        merchant_fees_expense,
        account_cash(),
        account_receivable(),
        account_fees_expense() );

LIST *tuition_refund_list_enrollment_list(
        tuition_refund_list );

LIST *tuition_refund_enrollment_list(
        tuition_refund );
```



## COURSE_DROP

```
/* Usage */
COURSE_DROP *course_drop_fetch(
        student_full_name,
        street_address,
        course_name,
        season_name,
        year,
        boolean fetch_enrollment,
        boolean fetch_offering,
        boolean fetch_course,
        boolean fetch_registration,
        boolean fetch_transaction );

COURSE_DROP *course_drop_new(
        ENTITY *student_entity,
        char *course_name,
        SEMESTER *semester );

/* Input */
ENTITY *student_entity;
char *course_name;
SEMESTER *semester;
ENROLLMENT *enrollment;
REGISTRATION *registration;
boolean refund_due;

/* Process */
TRANSACTION *course_drop_transaction(
        *seconds_to_add,
        student_full_name,
        street_address,
        transaction_date_time,
        enrollment->offering->course->program_name,
        enrollment->offering->course_price,
        enrollment->offering->revenue_account,
        account_payable() );
```